

Semantics and Optimizations for 2D Graphics

ANONYMOUS AUTHOR(S)

In recent years, significant advances have been made in numerical debugging and static analysis tools. Recent debugging tools significantly reduce overhead using double-double oracles, while recent static analysis tools accurately bound rounding errors using condition numbers. However, these new techniques also have downsides: double-double oracles can miss floating-point errors due to correlated errors, while condition number formalisms cannot cleanly handle over- and underflow. We show that combining both techniques avoids the downsides of each and implement this combination in EasterEgg. EasterEgg uses double-double computations but not as an oracle; instead, it detects erroneous operations using condition numbers, which minimizes overhead while achieving precision and recall similar to more expensive methods. EasterEgg also introduces a separate oracle to detect harmful over- and underflows more accurately; we implement this oracle in a new performant number representation. As a result, EasterEgg achieves a precision of 80.0% and a recall of 96.1% on a collection of 500 difficult numeric benchmarks. An ablation study shows that EasterEgg combines the strengths of both double-double oracles and condition numbers: it is more accurate than double-double oracles yet dramatically faster than methods based on arbitrary-precision condition number computations.

ACM Reference Format:

Anonymous Author(s). 2026. Semantics and Optimizations for 2D Graphics. 1, 1 (March 2026), 22 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Browser engines are an increasingly prominent way that users interact with applications—not only web applications like Office 365 or GMail but also desktop applications like Slack, Discord, and VS Code. Browser applications are graphical, and increasingly support complex graphical effects such as shadows, gradients, transparency, and animations. These features require high-performance to draw and redraw the application 60 or even 120 times per second to produce smooth animations and keep up with user interactions. A critical part of high performance is fast rasterization, the task of actually determining the color of each pixel drawn by the application, generally from a high-level description of shapes to be drawn and effects to be applied. A browser must typically do all this for millions of pixels in the span of hundreds of microseconds.

This requires highly-optimized rasterization libraries. The most prominent of these libraries is Skia. Skia is used by all prominent browser engines—it is the only rasterization library for the Google Chrome, Mozilla Firefox, and Ladybird browsers, and the Linux rasterization library for the Webkit engine behind Apple Safari. Skia is additionally used in the Android operating system, is one of two rasterization options in Flutter, a popular mobile application framework, and is also a popular backend for various libraries for rastering PDFs, images, and icons. Skia allows clients to describe shapes to be drawn, define how different shapes are blended together, and apply effects. Skia then converts this description into an efficient sequence of GPU shader calls using GPU backends tuned for various native APIs like Metal, Vulkan, and OpenGL. Skia is popular because it has wide support across many GPUs, and it is highly efficient.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

XXXX-XXXX/2026/3-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

However, to achieve max performance, clients must use Skia in the right way. Clients’ choice of API methods can cause Skia to allocate additional pixel buffers, perform additional blend operations, or switch from special-case to slower, general-purpose masking routines. Even sophisticated clients like Google Chrome generate subpar sequences of Skia commands due to the complexity of the Skia API and the clients’ own internal complexity. Moreover, less sophisticated clients have little hope of always performing the most optimal sequence of API calls and are therefore stuck with suboptimal rasterization performance. Ideally, these concerns would be handled internally, by an optimization pass within the Skia library, but to date no such optimization pass exists.

To address this, we develop μ Skia, a formalization of the Skia API. μ Skia captures key API features like canvas state, the layer stack, clipping, and blending. μ Skia is structured as a command language with an operational semantics defined in terms of a functional core; the functional core itself denotes to an abstract model of 2D graphics; and the abstract model permits multiple instantiations to allow reasoning module anti-aliasing and discretization error. The semantics, including various real-world extensions, is mechanized in Lean and used to prove various rewrite rules that serve to optimize common sub-optimal patterns of Skia commands generated by Google Chrome on real-world websites.

We then develop an optimizer for Skia using these rewrite rules. We demonstrate the optimizer on a collection of Skia programs generated by Google Chrome on the Alexa Top 100 websites. Considering the Skia programs that use complex blending effects, our optimizer is able to improve rasterization time on , with speedups ranging from to . The optimizer is also highly efficient and provides speedups even inclusive of optimization time. The optimizer is correct, with only relatively small pixel differences caused by anti-aliasing. Moreover, the optimizer produces optimization traces that can be loaded into Lean for translation validation against our semantics. Finally, the optimizer is effective across a variety of Skia backends and GPUs, and also speeds up rasterization on a second, “out of sample” set of Skia programs that the authors did not use when developing the optimizer.

The contributions to this paper are:

- (1) A new formalization of the Skia API (Section 3)
- (2) A collection of rewrites proven valid using this formalization (??)
- (3) A optimizer that speeds up real world Skia rasterization (Section 4).

2 SKIA

Pavel: make sure we dont reference colors

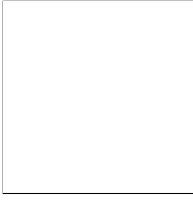
Skia is a popular library for 2D graphics. Skia’s basic APIs create drawable 2D “surfaces” and “layers”, draw shapes to those surfaces and layers, blend them together, and apply effects. These APIs are accessed by calling methods on a Skia SkCanvas object; however, internally, these SkCanvas operations just append commands to a command buffer, which we can take to be a stand-alone program. The program is executed when Skia’s flush method is called. This section introduces the language’s basic operations.

Bhargav: I need to get codex to read the skia source to make sure this is how it works

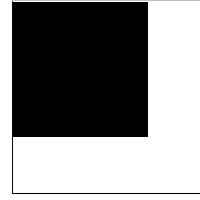
2.1 Drawing and Painting

The drawRect command is used to draw rectangles:¹

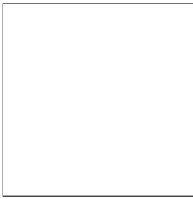
¹All code snippets in this section elide the SkCanvas object and method-call syntax to maintain focus on the Skia command language itself. We likewise drop the Sk prefix on most types.



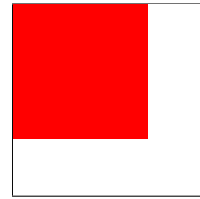
```
SkPaint p;
canvas->drawRect(rect1, p);
```



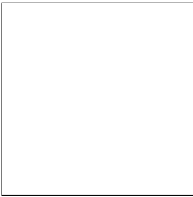
Here the layout is meant to evoke a Hoare triple: the left square shows the (empty) state of the canvas before the `drawRect` command, while the right square shows the state of the canvas afterwards. The `drawRect` command needs the bounds of the rectangle that needs to be drawn. `drawRect` also takes a `SkPaint` as an argument. A `SkPaint` describes *how* a geometry is drawn, for example how to *fill* the geometry. The default fill for a `SkPaint` is the color black, but we can also change it to another color.



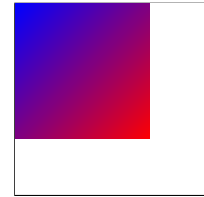
```
SkPaint p;
p.setColor(SK_ColorRED);
canvas->drawRect(rect1, p);
```



It's also possible to specify a non-constant fill, like a gradient:



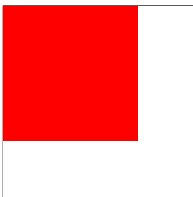
```
SkPaint p;
p.setShader(
    SkShaders::LinearGradient(...)
);
canvas->drawRect(rect1, p);
```



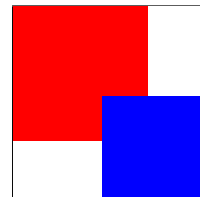
Skia also supports complex fills like images and shaders. Formally, filling is a stage of Skia's drawing pipeline that assigns a color to every point in the shape.

2.2 Blending

Things get a little more interesting once there is more than one shape being drawn. Suppose we draw a second, overlapping, blue square:

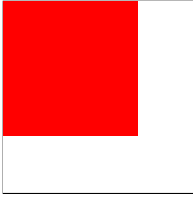


```
p.setColor(SK_ColorBLUE);
canvas->drawRect(rect2, p);
```

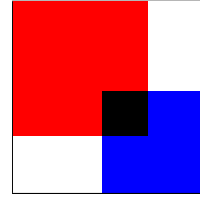


In Skia, the process of drawing this second shape involves, conceptually, two stages. First, as before, every pixel in `Rect2` is assigned a fill color, here a solid blue. Second, this assigned fill

color is *blended* with the existing color for that pixel. The default *blend mode*, called `SrcOver`, just chooses the new color,² but a program can select a different blend modes instead. For example, the `Multiply` blend mode combines both colors being blended:



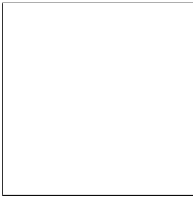
```
p.setColor(SK_ColorBLUE);
p.setBlendMode(
    SkBlendMode::kMultiply
);
canvas->drawRect(rect2, p);
```



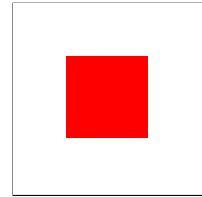
Now, in the intersection of the two rectangles, the initial red color is blended with the newly-drawn blue fill, which in the `Multiply` blend mode results in black. Formally, blending is another stage in the Skia drawing pipeline, which comes after filling and combines the fill color assigned to each pixel with the pre-existing color of that pixel.

2.3 Clipping

So far, `drawRect` has been applying the fill to the entirety of the shape being drawn. But, sometimes, we need more granular control over the drawing bounds. Skia provides a dedicated `clipRect` command to restrict or *clip* shapes to rectangles.³ One uses `clipRect` like this:

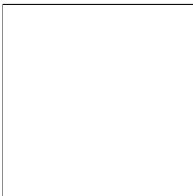


```
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
```

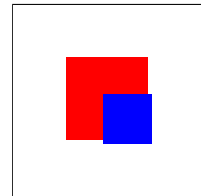


Note the unusual order of operations here. The `clipRect` command is run first, and it does not modify the image at all. Instead, it modifies the shape drawn by *later* draw operations. Formally, clipping is yet another stage of the Skia drawing pipeline; it comes before filling and modifies the shape being drawn.

Semantically, `clipRect` modifies the *clip state*, which stored in the canvas object and affects drawing commands. This means that a single `clipRect` operation can affect multiple `drawRect` operations:



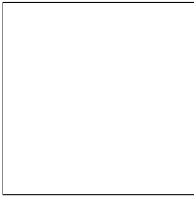
```
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
canvas->drawRect(rect2, bluePaint);
```



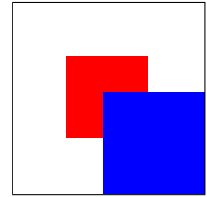
²In reality, the blending is more complicated and depends on the opacity or *alpha* of the new color, which can matter for anti-aliasing, but in our examples all the colors are opaque and we ignore anti-aliasing as a simplification.

³Skia allows clipping with arbitrary shapes, but the GPU back-end has fast paths for rectangles and rounded-rectangles, reducing them to scissor tests

Naturally, one might want to clip only a subset of drawing commands. Luckily, Skia maintains a *stack* of clip states; `clipRect` affects the top of the stack, which is used by drawing commands, while `save` and `restore` commands push and pop from the stack. `save` and `restore` act like brackets, restricting the effect of `clip` to the enclosed commands:



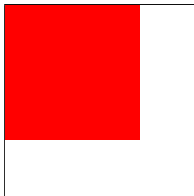
```
canvas->save();
canvas->clipRect(c_rect);
canvas->drawRect(rect1, redPaint);
canvas->restore();
canvas->drawRect(rect2, bluePaint);
```



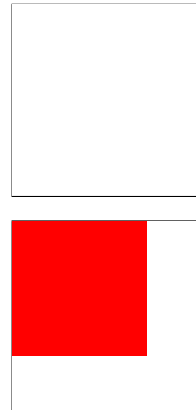
2.4 Layers

Pavel: I thought this was the most abrupt and unmotivated transition. Not the end of the world.

Much like `save` and `restore` can save clip state, there is also a `saveLayer` and `restore` command that saves and restores the drawing surface itself. Conceptually, `saveLayer` allocates a new, empty drawing surface.



```
canvas->saveLayer();
```



All drawing commands until the corresponding `restore` draw to this new surface.

canvas->drawRect(rect2, bluePaint);

Then, when the restore command it reached, the whole resulting drawing surface is blended back into the original drawing surface.

restore();

Semantically, you can think of SaveLayer commands as re-associating the order of blend operations. In the Save example, the order of operations is

blend(blend(empty, red), blue),

While in the SaveLayer example the order of operations is

blend(blend(empty, red), blend(empty, blue)).

With SrcOver blending, which is associative, this order of operations is not important, but with other blend modes it becomes significant. For example, we repeat the previous snippet, but pass a Multiply blend mode to the saveLayer command.

295

296

297

298

299

300

301

302

303

304

305

306

307

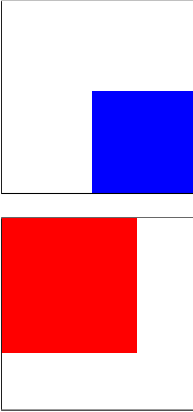
308

309

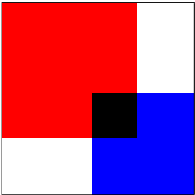
310

311

312



```
restore(); // multiply blending
```



2.4.1 *Masking*. To further motivate the need for `saveLayers`, consider masking again. In Section 2.3 we saw how we can use `clipRect` to mask images. This works when we only need to mask by a rectangle. But masking with complicated shapes is hard with just clipping. Instead of trying to clip directly with a complicated shape, we can instead draw the clip shape in stages on a separate layer and mask using that shape.

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

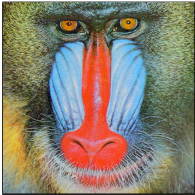
339

340

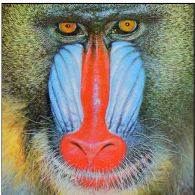
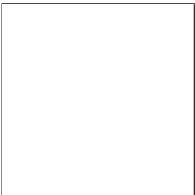
341

342

343

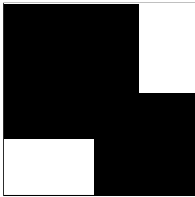
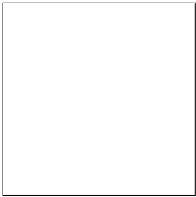


```
canvas->saveLayer();
```

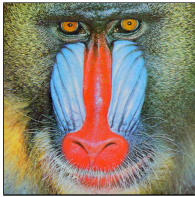


We now draw the mask shape on the new layer.

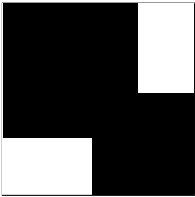
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392



```
canvas->drawRect(rect2, bluePaint);
```

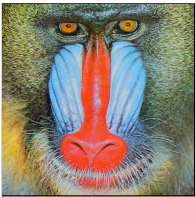


DstIn emulates masking through blending; When we DstIn one pixel over another, an opaque top pixel keeps the bottom pixel, and a transparent top pixel removes it.

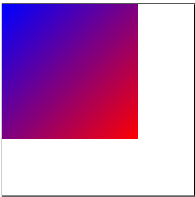


```
restore(); // multiply blending
```

[masked_mandrill.pdf](#)

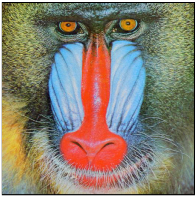


DstIn blending is also far more powerful than just clipping, because it also allows masking with gradients, allowing us to create more complicated images that just clipping would not allow us to. For example, here is the previous image, now masked with a radial gradient.



```
restore(); // multiply blending
```

[grad_mandrill.pdf](#)



Pavel: I think we require a special
Transparent color.

Image : Point $\rightarrow$ Color

Shape : Point $\rightarrow$ Bool

Blend : Color $\times$ Color $\rightarrow$ Color

Filter : Color $\rightarrow$ Color

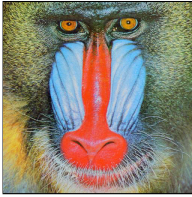
Solid(Color) | LinearGradient(...) ... : Image
Rect(...) | RRect(...) | Path(...) | Text(...) : Shape
Id | Luma ... : Filter
SrcOut | DstIn | ... : Blend

Fig. 1. Our abstract model of Images and Shapes is built from abstract types Pixels and Colors.

Fig. 2. Layer provides a simple functional description of Skia images.

2.5 Equivalences

In ?? we saw how complex masks can be done with ease using a saveLayer with DstIn blending. But what if just a rectangle mask is needed? Our saveLayer and DstIn program would still work. But, it is clearly less performant than just doing a clip; We have to allocate a whole new layer, just to do something Skia already has a fast path for! Here, we have two different skia programs that produce the same image, but one is clearly a better choice than the other.



```
restore(); // multiply blending  
restore(); // second program
```

simple_mandrill.pdf

Therefore it should be possible to replace a skia program with a faster one, given they have identical results. In Section 3 we develop a semantics, which allows proving that two Skia programs have identical results, including verifying general-purpose rewrite rules. Section 4 then uses these rewrites to build a general-purpose Skia optimizer that rewrites programs to reduce the use of multiple layers. Section 5 then shows that this compiler results in dramatically faster rendering on a wide range of real-world Skia programs.

3 SEMANTICS

This paper's semantics of Skia consists of three components. The bottom-most layer is an abstract model of images, shapes, blends, and filters defined in terms of abstract types Point and Color, which can be instantiated to model as much of the actual Skia API as necessary. The middle layer is functional operations Draw and BlendLayer and an abstract type Paint, which denote to our abstract model. The top layer is an imperative language, μ Skia, whose operational semantics provides a model of Skia. This section describes all three layers and then examines 4 rewrite rules proven valid using the semantics.

3.1 Abstract Model

Traditional programming language semantics typically do not have to define their abstract model, since it is typically based on commonly-known types such as pairs, functions, and integers. In the graphics domain, the core types are not so well established, so the abstract model must be explicitly defined. The abstract model of our semantics is shown in Figure 1; it is based around

$$\begin{aligned}
& \llbracket \text{Empty}() \rrbracket(\text{pt}) = \text{Transparent} \\
& \llbracket \text{BlendLayer}(l_{\text{bot}}, l_{\text{top}}, p) \rrbracket(\text{pt}) = \text{blend}_p(\llbracket l_{\text{bot}} \rrbracket(\text{pt}), \text{filter}_p(\llbracket l_{\text{top}} \rrbracket(\text{pt}))) \\
& \llbracket \text{Draw}(l, s, p, c) \rrbracket(\text{pt}) = \text{blend}_p\left(\llbracket l(\text{pt}) \rrbracket, \text{filter}_p\left(\begin{cases} \text{fill}_p(\text{pt}) & \text{pt} \in s \cap c \\ \text{Transparent} & \text{otherwise} \end{cases}\right)\right)
\end{aligned}$$

Fig. 3

two core abstract types, Point and Color. The Point and Color types are then used to define the key types in Skia. An Image is defined as a Color for every Point while Shapes are defined as predicates on Points. Equivalence of two Images is defined as assigning the same Color to each Point. Additionally, since a major focus of this semantics is reasoning about color blending and filtration, types for Blends and Filters are defined as binary and unary operators on colors.

This abstract model hides substantial complexity in the actual implementation.⁴ The actual Skia implementation defines Point to be a pair of 32-bit signed integers and Color to be a 32-bit unsigned integer interpreted as ARGB pre-multiplied byte-sized color,⁵ and operations on points and colors must thus be anti-aliased and rounded. The exact anti-aliasing and rounding behavior is not guaranteed by Skia and is also exceptionally complex, including hardware dependence in some back-ends; the abstract model allows our semantics to ignore these details. The equivalences proved are thus *up to* anti-aliasing and rounding details (and also assume away issues like coordinate overflow).

Matching the abstract model to Skia would require actually defining the vast array of Images (solid color fills, gradients, image files), Shapes (rectangles, paths, text outlines), Blends (roughly two dozen), and Filters (in fact, these can be defined by shader programs in a custom SkSL shader language). Defining these objects and their equivalences would be a complex undertaking, especially given the pace of change in Skia itself. Luckily, the compiler described in Section 4 does rather superficial reasoning over these objects and only special-cases a handful of them. Our Lean formalization only defines those special cases. The abstract model nonetheless allows for extensibility; for example, this section does not discuss opacity, but our Lean formalization does include it. This extension does not require any changes to the abstract model, only the addition of specific color and blend operators, and thus re-uses the denotational and operations semantics defined in the remainder of this section.

3.2 Skia as Layers

Skia does not permit arbitrary drawing and blending operations; instead, these always happen in a prescribed order. We thus introduce the language of Layers, shown in Section 3.3, which defines the core Draw and BlendLayer operations in Skia. We focus on these two operations specifically because they are the core target of the optimizing compiler described in ???. The Layer constructors can be defined in terms of the abstract model, as shown in Figure 3.

⁴In the same way that a traditional programming language semantics might hide the complexity of actually storing closures on the heap.

⁵“ARGB” gives the in-memory byte order of the alpha, red, green, and blue components of a color. The interpretation of these bytes as an unsigned 32-bit integer will depend on the endianness of the platform itself. The colors are premultiplied, meaning that a color with an alpha value of α will have its red, green, and blue components range from 0 to α ; this form makes for somewhat more efficient computation.

```

491
492 Command c ::= Draw(g: Shape, p: Paint)
493 Layer ::= Empty() | Draw(l: Layer, s: Shape, p: Paint, c: Shape) | Clip(cl: Shape)
494 | BlendLayer(l_bot: Layer, l_top: Layer, p: Paint) | SaveLayer(p: Paint); c; Restore()
495 | Save(); c; Restore()
496 | SaveLayer(p: Paint); c; Restore()
497 | c; c
498
499 Paint ::= Paint(fill: Image, filter: Filter, blend: Blend)

```

Fig. 4. Layer provides a simple functional description of Skia images.

Skia has two core operations: drawing a shape onto an image, and blending two images (representing two layers) into one. Each operation is parameterized by a Paint object, which indicates the current blend mode, the fill type, and so on; drawing also requires the clip region from the canvas state. The signatures for each operation are defined in Section 3.3 as constructors of a Layer type. Equivalence of two Layer terms is defined by equivalence of their denotations to Images.

The operations can be denoted to the abstract model, as shown in Figure 3. Empty has a straightforward denotation to an all-Transparent image. BlendLayer is likewise straightforward, applying a blend operator, determined by the paint p , point-wise on the two layers being blended. Draw is most complex. Conceptually, it first performs clipping, constructing the intersection $s \cap c$ of the drawn shape and the clip state; then it performs filling, computing the fill color for each point in the shape; and finally it performs blending on the original color and computed fill for each point. The blend is performed even for points outside $s \cap c$; this is critical for, for example, DstIn blending. Note that the equalities over denotations are compositional. That is, if we know that $\llbracket l \rrbracket = \llbracket l' \rrbracket$, then $\llbracket m \rrbracket = \llbracket m[l/l'] \rrbracket$. The fact that denotations are composable means we can use equalities over denotations as rewrites, enabling us to actually use our semantics to verify optimizations.

3.3 μ Skia

μ Skia is a simplified version of the Skia API. A μ Skia program consists of a sequence of commands whose syntax is defined in Figure 4. μ Skia includes the critical features described in Section 2, such as the Draw command for drawing, the Clip command for manipulating the clip state, and the Save, SaveLayer, and Restore stack manipulation commands, μ Skia diverges from the full Skia API in two main ways. First, the syntax definition is more regular to eliminate unimportant special cases: Save, SaveLayer, and Restore must be properly bracketed, and the Draw and Clip commands take arbitrary Geometries. Second, only a small fraction of the API is modeled. Nonetheless, extending μ Skia to cover more of the API is straightforward, and our Lean formalization includes additional features such as opacity, styles, and transforms.

μ Skia programs manipulate a state Σ :

$$\Sigma : \text{List}[\text{Layer}] \times \text{List}[\text{Geometry}]$$

The first list in Σ is the stack of layers manipulated by SaveLayer, while the second list is the stack of canvas states manipulated by Save. In this paper, the canvas state is a single Geometry because the only canvas operation is Clip, but in our Lean formalization the canvas state includes additional fields. As is standard, we treat the head of the list as the top of the stack. The initial state is $\langle [\text{Empty}()], [\mathcal{U}] \rangle$, an empty layer and the full clip state, each as single-element stacks. A μ Skia program then performs a sequence of steps $(c, \Sigma) \mapsto \Sigma'$, after which the final state consists of a single layer L , which is the output image, and some clip state, which is discarded.

We now describe the core steps in the operational semantics. The simplest is the Draw command, which modifies the top Layer:

$$\frac{}{\langle \text{Draw}(g, p), \langle l :: L, c :: C \rangle \rangle \mapsto \langle \text{Draw}(l, g, p, c) :: L, c :: C \rangle}$$

Note that the Draw operation on the top Layer includes both the Geometry and Paint from the Draw command as well as the current clip from the canvas state:

The Clip command likewise modifies the top canvas state, intersecting it with the new clip geometry:

$$\frac{}{\langle \text{Clip}(c'), \langle L, c :: C \rangle \rangle \mapsto \langle L, c \cap c' :: C \rangle}$$

Note that the full Skia API includes a SkClipOp parameter that allows instead taking the difference between the current and new clip; this paper does not show that (straightforward) extension but our Lean formalization does include it.

The Save command duplicates the top canvas state. Then, when its matching Restore is reached, that top canvas state is discarded:

$$\frac{\langle \text{cmd}, \langle L, c :: c :: C \rangle \rangle \mapsto \langle L', c' :: c :: C \rangle}{\langle \text{Save}(); \text{cmd}; \text{Restore}(), \langle L, c :: C \rangle \rangle \mapsto \langle L', c :: C \rangle}$$

Note that it does not make any changes to the layer state.

The SaveLayer command, meanwhile, allocates a new, empty layer for the top of the layer stack and, when the Restore command is reached, blends that new layer with the one below:

$$\frac{\langle \text{cmd}, \langle \text{Empty}() :: l :: L, c :: c :: C \rangle \rangle \mapsto \langle l' :: l :: L, c' :: c :: C \rangle}{\langle \text{SaveLayer}(p); \text{cmd}; \text{Restore}(), \langle l :: L, c :: C \rangle \rangle \mapsto \langle \text{BlendLayer}(l, l', p) :: L, c :: C \rangle}$$

The inference rule for sequential composition is straightforward and, along with various extensions to μSkia , is given in our Lean formalization, which we intend to submit as an artifact.

The μSkia semantics is deterministic, and programs never get stuck. Equivalence of μSkia programs, which we notate $=_{\text{Skia}}$, is defined as equivalence of their final images. This equivalence relation is *not* closed under composition or substitution in general, but it is closed under substitution under SaveLayer commands.

3.4 Rewrites in μSkia

This section provides a number of case studies, using the μSkia semantics to prove equivalences between μSkia programs. It demonstrates that the μSkia semantics are sufficiently powerful to define and prove valuable optimizations on real-world Skia programs. Specifically, we collected the Skia programs used by Google Chrome 140.0.7339.16 to render the Alexa Top 100 websites and examined them to find common, suboptimal patterns. In general, the suboptimality comes from using more SaveLayer commands than necessary, and thus both using more GPU memory and also performing more blends than would otherwise be required. To demonstrate the flexibility of the μSkia semantics, this section shows both general-purpose rewrites that apply to a vast range of Skia programs as well as more specific rewrites that we expect to apply only to Skia programs produced by Google Chrome or only to specific web frameworks or even websites.

3.4.1 SrcOver SaveLayers. SrcOver is by far the most common blend mode, corresponding to the normal intuitive meaning of “drawing over” some other image, and SaveLayers with SrcOver blend mode are often unnecessary, suggesting the following idealized equivalence:

```

... // l1                ... // l1
SaveLayer(Paint(p, SrcOver));   =Skia  Save();
... // l2                ... // l2
Restore()                    Restore()

```

Specifically, this equivalence holds when the commands labeled ℓ_2 all use SrcOver blend mode.⁶ Note that this equivalence replaces the SaveLayer command by an equivalent Save command instead of removing it entirely. This is necessary to allow commands in ℓ_2 to change the canvas state. Save is much cheaper to execute than SaveLayer, since it does not require allocating a new GPU surface, so this equivalence still, in practice, significantly cuts computation time.

This equivalence is proven by induction over commands in ℓ_2 . It is convenient to assume that sequential composition is left-associated (we do this in our Lean formalization)

Pavel: Do we?

; this means the induction effectively proceeds by “peeling off” the last drawing command in ℓ_2 . After expanding out the μ Skia semantics, the inductive step requires proving the following equivalence of Layer terms:

```

BlendLayer(
  l1
  Draw(l2, g, (SrcOver), c),
  (srcOver))
=Skia
Draw(
  BlendLayer(l1, l2 p),
  g, p, c)

```

Note the Draw command uses a SrcOver blend; this is why the commands in ℓ_2 must all use SrcOver blends. Applying this rewrite once “peels” a single Draw command out of the inner SaveLayer. Applying it repeatedly results in the original equivalence of μ Skia programs.

This equivalence is, in turn, equivalent to the following equality in our abstract model, which asserts that SrcOver blends are associative:

$$\text{SrcOver}(a, \text{SrcOver}(b, c)) = \text{SrcOver}(\text{SrcOver}(a, b), c)$$

In our Lean formalization, we provide an instantiation of our abstract model using real-valued coordinates and real-valued colors, and prove this equivalence (including in the case of partial transparency). There is also a base case to the recursion, involving a BlendLayer of an Empty Layer. One could optionally prove the case of a BlendLayer of a BlendLayer Layer, but we did not see such Skia programs generated by Chrome.

3.4.2 Gradient Masks. A more specialized rewrite concerns the use of gradients as masks and is common on Russian websites like mail.ru, dzen.ru, and yandex.ru. All three of these sites seem to use a common web framework that seems to result in Chrome generating the following optimizable Skia code:

```

SaveLayer(p);
Draw(solid fill, srcOver);
SaveLayer(DstIn);
Draw(Gradient(opaque));
Restore();
Restore();
=Skia
SaveLayer(p);
Draw(solid fill, srcOver);
Restore();

```

Pavel: What is with this example, why is all the syntax fake??

On the left hand side, a gradient is used to draw the mask image. However, with DstIn blending, only the opacity (alpha) of the mask is used, not its color, and in many cases the entirety of the

⁶If we also include opacity, then we must also restrict the savelayer to be opaque. We have formalized this restriction in our mechanization.

gradient is opaque. As a result, the mask operation is entirely superfluous. It is not clear to use why this pattern is used; possibly the gradient could be configured to be partially transparent but isn't on these websites.

No induction is needed in this example, since only a fixed number of Commands are present. Using the μ Skia examples, this rewrite unfolds to:

Pavel: Show the Layer formula

This reduces to the following equation in the abstract model:

$$\text{DstIn}(\text{SrcOver}(\text{Transparent}, a), b) = \text{SrcOver}(\text{Transparent}, a)$$

where b is known to be opaque, which it is because all of the stops of the gradient are themselves opaque.⁷ Our Lean formalization proves this formula correct for our real-valued instantiation of the abstract model.

3.4.3 Subsuming Filters. SVG are the primary way to draw graphics on the web. The SVG standard has drawing commands just like Skia. Consider the following SVG drawing and its rendered output.

```
<svg width="80" height="30" viewBox="0 0 80 30">
  <defs>
    <radialGradient id="b" ...>...</radialGradient>
    ...
    <pattern id="x" ...>
      <rect width="100" height="100" fill="#7D2AE7"/>
    ...
  </pattern>
</defs>
<mask id="a" ...>
  <path fill="#fff" d="M79.444 18.096 ... 69.192 21.888z"/>
</mask>
<rect mask="url(#a)" width="100" height="100" fill="url(#x)"/>
</svg>
```



Fig. 5. SVG snippet used in this subsection, shown alongside its rendered result.

This SVG program, in short, fills a rectangle with a complicated gradient. It then masks in the Canva logo onto the gradient. This is a pattern we have seen before in Section 2. Specifically, the mask is implemented using a savelayer with a dstin blend, when the svg is lowered to Skia commands. But, the conversion is not that straightforward. The default masking method for SVG drawings is luminance masking. That is, white pixels(the brightest color) masks in and black pixels(the darkest color) masks out. If we recall dstin blending, dstin blending is opacity based. So we need to convert white pixels into opaque pixels, and black pixels to transparent pixels. Skia does this using a luminance filter.

...		...
draw		draw
savelayer dstin		savelayer dstin
savelayer lumafilter	= _{Skia}	savelayer dstin
draw mask white		draw mask opaque
restore		restore
restore		

⁷For more complicated fills like images, we could pessimistically assume them to not be opaque. In principle, more precise reasoning might be possible, such as reading the Graphics Control Extension block Transparent Color Index for GIF-formatted images.

Now if the mask is simple i.e. we only mask one shape and not complex intersection of many shapes, we should be able to push in filter into the color of the mask. Which is why our rewrite removes the savelayer filter, while also chaing the color of the mask. We can reduce our μ Skia rewrite into a layer rewrite, and we have to prove this statement.

$$srcover(l1, luma(srcover(Transparent, fill))) = srcover(l1, srcover(Transparent, luma(fill)))$$

We can prove this equality correct, because we can prove that srcovering a pixel over the transparent pixel is equivalent to just drawing the pixel. One thing we elide, is the case when we are out of the shape we are trying to draw. Then our equation looks kind of like this

$$srcover(l1, luma(srcover(Transparent, Transparent))) = srcover(l1, srcover(Transparent, luma(Transparent)))$$

So it is important to note that we can only move filters which have the Transparent pixel as the identity pixel. This is true for the luminance filter that Skia emits.

3.4.4 *Dstin to Mask*. After performing the above rewrite we are left with

```
...
draw
savelayer dstin
draw mask opaque
restore
```

We have already seen this rewrite before in Section 2.

```
...
draw
savelayer dstin   =Skia  clip mask
draw mask opaque  draw
restore
```

While we only asserted its correctness in Section 2, we can actually prove it with the semantics now. Let us first reduce this rewrite to μ Skia terms.

```
blendlayer(
  1,
  draw(Empty, g, p, c), =Skia clipAll(1, g  $\cap$  c)
  dstin
)
```

clipAll is a recursive function over layers, which propogates a clip across a sequence of draws. Note that we will be referring to $g \cap c$ as mask going forward. We define it as such;

```
clipAll(Empty, m) = Empty
clipAll(Draw(1, g, p, c), m) = Draw(clipAll(1, m), g, p, c  $\cap$  m)
```

The proof of the rewrite now follows through induction over the layer l . For the base case, where l is Empty, the rewrite reduces to

$$dstin(\llbracket Empty \rrbracket, \llbracket Draw(...) \rrbracket) = \llbracket clipAll(Empty, mask) \rrbracket$$

If we recall how dstin blending works, dstin blending in a opaque pixel masks in, while a transparent pixel masks out. Now if the pixel we are blending on is Transparent – which $\llbracket Empty \rrbracket$ is – the result remains transparent. The other side of the equality also reduces to Transparent by definition of clipAll.

Now for the base case, $l = Draw(l', g', p, c')$. Which means we need to prove:

$$\begin{aligned} &dstin(\llbracket Draw(l', g', p, c') \rrbracket, \llbracket Draw(...) \rrbracket) \\ &= \llbracket clipAll(Draw(l', g', p, c'), mask) \rrbracket \end{aligned}$$

We now perform case analysis on the location of point to be drawn (remember that layer equalities are quantified over points). When the point to be drawn is outside the mask, we get

$$\begin{aligned} &dstin(\llbracket Draw(l', g', p, c') \rrbracket, Transparent) \\ &= \llbracket clipAll(Draw(l', g', p, c'), mask) \rrbracket \end{aligned}$$

As *dstin* masks out Transparent pixels, and following the definition of *clipAll*:

$$\begin{aligned} &Transparent \\ &= \llbracket Draw(clipAll(l'), g', p, c' / capmask) \rrbracket \end{aligned}$$

As the point inside is outside the mask, it is outside the clip, hence proving the equality.

Now we turn to the case where the point to be drawn is outside the mask. Looking at the inductive hypothesis.

$$dstin(\llbracket l \rrbracket, Draw(...)) = \llbracket clipAll(l, mask) \rrbracket$$

We know that we are masking with a opaque pixel, meaning

$$\llbracket l \rrbracket = \llbracket clipAll(l, mask) \rrbracket$$

We have to prove:

$$\begin{aligned} &dstin(\llbracket Draw(l', g', p, c') \rrbracket, \llbracket Draw(...) \rrbracket) \\ &= \llbracket clipAll(Draw(l', g', p, c'), mask) \rrbracket \end{aligned}$$

Again we are masking with an opaque pixel, so this reduces to

$$\begin{aligned} &\llbracket Draw(l', g', p, c') \rrbracket \\ &= \llbracket clipAll(Draw(l', g', p, c'), mask) \rrbracket \end{aligned}$$

By the definition of *clipAll* we get

$$\begin{aligned} &\llbracket Draw(l', g', p, c') \rrbracket \\ &= \llbracket Draw(clipAll(l', mask), g', p, c' \cap mask) \rrbracket \end{aligned}$$

Using the inductive hypothesis and the compositionality of our denotations:

$$\begin{aligned} &\llbracket Draw(l', g', p, c') \rrbracket \\ &= \llbracket Draw(l, g', p, c' \cap mask) \rrbracket \end{aligned}$$

as we know that *pt* is inside the mask, so $c' \cap mask = c'$ finishing the proof.

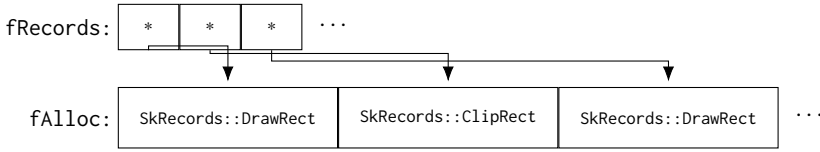


Fig. 6. SkRecord stores an array of pointers (fRecords) into a contiguous chunk of record objects (fAlloc), each of which represents a single Skia command.

4 COMPILER

To show that the μ Skia semantics are useful for developing impactful Skia tooling, this section discusses the implementation of a Skia optimizer using the rewrite rules discussed in ?? . Besides correctly implementing the relevant rules, the optimizer must also be highly-efficient because Skia rasterization must meet a refresh rate target and thus has a budget of hundreds to thousands of microseconds, of which the optimizer must be a smaller share. For this reason, the optimizer sometimes implements a somewhat-restricted version of the rewrites to ensure that it runs a single pass over the Skia command buffer.

4.1 Memory Management

When a Skia client calls a methods on the SkCanvas object, Skia records this command in an in-memory buffer called an SkRecord. SkRecord is a dense collection of SkRecords objects.⁸ Each SkRecords object, for example SkRecords::DrawRect, is distinguished by a tag and then has additional fields for all the arguments to that specific command. SkRecords::DrawRect, for example, will contain a reference to a SkRect object which defines the bounds of the rectangle being drawn and an SkPaint. The SkRecord manages a buffer, which is an array of pointers to SkRecords objects, each of which is allocated in an SkArenaAlloc arena allocator. Figure 6 illustrates the memory layout. Importantly, this design means that the compiler can mutate the SkRecord buffer without moving the actual SkRecords objects.

SkRecord provides two methods to access the underlying SkRecords buffer, visit and mutate. Both methods also need a functor⁹ In general, these functors allow matching and also mutation, but we only use the SkRecords::Is<...> functor, which performs tag-checking and compiles to an efficient comparison while staying type-safe.

Skia can serialize SkRecord objects to and from a binary format called a “Skia Picture” or skp. All information in a SkRecord format required for drawing, including images, are serialized into a skp.¹⁰ This allows us to evaluate our optimizer without interference from the complex Google Chrome runtime; specifically, our evaluation reads an skp from disk, optimizes it, and writes the optimized skp to a new file. Note that though the input and optimized output are stored in different files, during optimization the optimizer mutates a single SkRecord without making any copies.

Importantly, Google Chrome can dump its generated Skia programs to disk as skps with the chrome.gpuBenchmarking.printToSkPicture debug method, available when Chrome is run with -no-sandbox and -enable-gpu-benchmarking flags. We use this facility to collect benchmarks for our evaluation, as discussed in more detail in Section 5. It is important to note here that Chrome generates multiple Skia programs, meaning multiple skps, for a single website. These multiple programs represent different “layers” of the page, which are then composited in a later step; this split

⁸Confusingly, the collection of the many SkRecords objects uses the singular word SkRecord.

⁹Functor here refers to a C++ concept—a class that has the function call operator overloaded—not the unrelated concept in category theory.

¹⁰An exception seems to be that typefaces do not gets serialized.

into layers is important for efficient animations [?]. The optimizer operates on layers individually, and neither performs cross-layer optimizations nor influences how Chrome splits a page into layers.

4.2 Rewriting

Applying the rewrite rules in ?? requires change, remove, or add commands in the command buffer. Changing commands is easy: the optimizer allocates a new SkRecords object in the arena and changes the buffer to point to this new object. Removal likewise change the command to remove into a special SkRecords::NoOp command. Insertion, however, is tricky, because the input command buffer is dense. Inserted commands are allocated into the arena as normal. However, to minimize run time, the actual insertion into the buffer is delayed. Instead, all insertions are first made into a separate insertion buffer. Then, after the compilation pass is complete, a new buffer of the right size is allocated and a single linear scan is performed to merge the original buffer and the insertion buffer. We learned of this technique from the WebKit browser engine's B3 JIT, where it is called an "InsertionSet" [1], though we believe this technique is well-known.

This all is not super useful. We do want the struct MatchState, do show that, maybe with a "..." for the actual states. I'm confused about int index vs the index_stack, what's the deal there? And then instead of the right hand side of this figure, we want the per-layer state machine for matching rewrite rules. Just draw one for each rule.

```

struct RemoveOpaqueSaveLayers {
    void transform(SkRecord* records);

private:
    struct MatchState {
        enum class { Matching, Ignore } state;
        int index;
        int saveCount;
    };
    skia_private::STArray<8, MatchState> state_stack;
    skia_private::STArray<8, int> index_stack;

    SkRecords::Is<SkRecords::SaveLayer> isSaveLayer;
    SkRecords::Is<SkRecords::Save> isSave;
    SkRecords::Is<SkRecords::Restore> isRestore;
    SkRecords::IsSingleDraw isDraw;
};

void transform(SkRecord* records) {
    for (int i = 0; i < records->count(); i++) {
        if (records->mutate(i, isSaveLayer)) {
            // ...
        } else if (records->mutate(i, isSave)) {
            // ...
        } else if (records->mutate(i, isDraw)) {
            // ...
        } else if (records->mutate(i, isRestore)) {
            // ...
        }
    }
}

```

Fig. 7

The remaining complexity of the optimizer consists of efficiently finding the relevant patterns in the command buffer. To maximize performance, it is important that each compilation pass performs a single pass over the command buffer. Each pattern is thus matched via a state machine stored on a small stack; the stack of state machine states is stack-allocated for typical skps thanks to a Skia's STArray container.¹¹ Every command pushes onto the state machine stack, and every matching pops from it; thus, every layer is matched independently, though for some rewrite rules commands in one can change state in lower layers. Since Skia uses the same command for both and , the state machine state also stores a save counter to match them properly. The index of the command is also stored, for rewriting. All commands advance the state machine.

¹¹Chrome typically does not deeply nest commands, so in practice no heap allocation is needed in the pattern matcher.

Figure 7 shows one such state machine for the SrcOver rewrite in `??`. For this rewrite, the state machine starts in a matching state if the has a SrcOver blend mode (and is opaque; our Lean formalization includes opacity) and transitions to a non-matching state if any non-SrcOver commands are observed. commands also transition the parent layer to a non-matching state; in theory this could be supported but we didn't see this pattern in any of our evaluation programs. Once a matching a is seen, if the current state is matching, the actual rewriting is performed by changing the command into a command. Figure 7 shows state machines corresponding to the other rewrites of `??` as well; while more complex, these are straightforwardly derivable from the rules themselves.

5 EVALUATION

Bhargav: need to workshop this intro

We evaluate the optimizer, and by extension the semantics, by asking four research questions:

RQ1 Are the optimizations correct?

RQ2 Does the otpimnizer improve the render time of real world Skia programs?

RQ3 Does the optimizer pay for itself?

RQ4 Are the optimizations generic across websites, GPUs, and backends.

5.1 Methodology

We evaluate our optimizer on the top 100 most-visited websites in the world.¹² This data is collated from rankings from Semrush [cite](#) and Similiarweb [cite](#). As our optimizer runs on `skp` files, we first dump them from Chrome, as described in [foo](#). We reiterate, that one website can yeild more than one `skp` file. As our rewrites focus on removing `SaveLayers`, we remove `skps` that are do not have the `SaveLayer` commmand. This filtering is enabled by the `skp_parser` tool that converts `skp` files to JSON files, for easily manual or scripted inspection. As browsers run on different architectures with different GPU API, we also run our optimizer on different machines. For this evaluation we run our optimizer on two machines whose specs we list below:

Machine #1 8-core Apple M2 CPU/GPU, 16GB RAM, clang version 17.0.0

Machine #2 i7-8700K CPU, Intel CoffeeLake-S GT2 GPU, 32 GB DDR4 RAM, clang version 18.1.3

It is important to note that the benchmark set, that is the number of layers of all 100 websites, differ from machine to machine. This mainly due to Chrome acting differently with respect to compositing(the step before rendering wherein Chrome splits visual elements on websites into layers) from machine to machine. So for the evaluation, we have a total of 99 benchmarks for machine #1 and [foo](#) benchmarks for machine #2.

The optimizer is based off of Skia commit `f931a2e13749b3c95c76e57ba2589c1abb6b6192` of the main branch [cite](#). We describe research qeustion-specific methodology in the relevant subsections.

5.2 RQ1

5.3 RQ2 Does the otpimnizer improve the render time of real world Skia programs?

To answer this research question, we benchmark the rendering time of both the unoptimized and optimized `skps` of the websites we discussed before. To measure render time, we use `nanobench`, a benchmarking tool internal to Skia. `nanobench` actually has many different benchmarking options, the `skp` playback options are what we use to measure render time. As rendering time is not many magnitudes greater than the timer overhead¹³ we need to choose a careful ebcnhmarking scheme

¹²We do not run it on the full list , as we exclude explicit websites from the list.

¹³`nanobench` uses `std::chrono::steady_clock` as its timer.

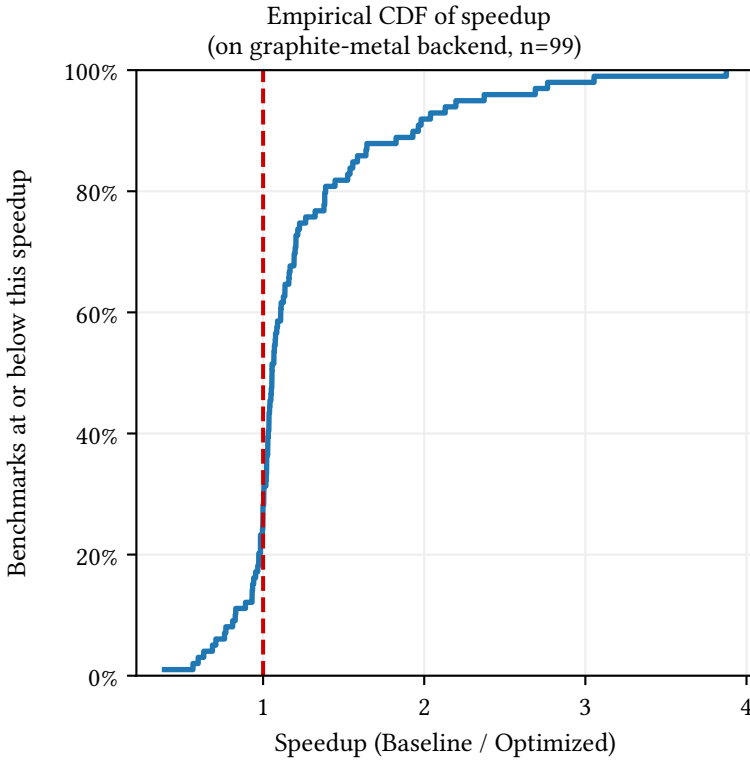


Fig. 8

that is able to factor out timer overhead. We now describe how nanobench avoids that, by talking about the measurement scheme. Before actually measuring the time taken to render for a specific skp file, nanobench first measures the number of loops of rendering needed to make the timing overhead negligible. In nanobench, each reported sample is computed as: total elapsed time for a run, divided by the number of playback loops in that run. So the key to making the timer overhead negligible is to choose a good loop count. For GPU playback, nanobench calculates the number of loops needed so each sample lasts about a target duration (default `-gpuMs=5 ms`): it doubles loops until the sample exceeds the target, then rescales loops to land near that target. This makes each sample long enough that timer-call overhead is negligible. Using this scheme, we measure 100 samples for each benchmark, both the optimized and unoptimized skp. We summarize the rendering speedup across 100 samples using their geometric mean. For this research question we only measure rendering time of all benchmarks on Machine #1 running Skia's Graphite/Metal GPU backend. Graphite is Skia's new GPU rasterization backend, rebuilt for modern graphics APIs (like Metal, Vulkan, and D3D12) with a cleaner architecture and multithreaded rendering workflow. Skia's older GPU backend, Ganesh, targeted GPU's through GL. While this section only discusses performance on Graphite, in `foo` we also show our optimizer's performance on Ganesh as well as other hardware platforms in `foo`.

The CDF chart in `foo` plots the optimizer's results. The X axis of the plot is the speedup –baseline render time / optimized render time– while the Y axis is the percentage of benchmarks that have speedup less than or equal to a certain speedup. On this specific machine-backend combination,

we see a maximum speedup of `foo`. Moreover Looking at `foo`, note that most of the CDF line is over the $1\times$ speedup line. This means a majority – here `69.7%` – of the benchmarks render faster after being optimized.

The long right tail of the CDF plot is due the optimizer significantly improving the rendering time of group of russian websites we listed in `foo`. Recall, that these russian websites seem to share the web framework, and as a result we are able to remove the redundant gradient mask layers described in `foo`. Moreover, these gradient mask layers are always inside an opaque save layer, which means that the gradient mask layer encovers a new rewrite opportunity.

Slowdowns are `foo` of the benchmarks. All of the slowdowns are seen on website with *no matches*, suggesting all of the slowdowns seen in the left-tail of the CDF are all measurement/runtime noise and not caused by our optimizer.

Furthermore, all of the significant speedups we see in the CDF plot are due to the optimizer, and not measurement/runtime noise. Specifically, out of the `foo` benchmarks, `foo` are speedups. Within these speedups, `foo` are actually optimized by the optimizer. The range of the speedups of the optimized benchmarks – `foo` to `foo` – are larger than the range of the speedups of the unoptimized benchmarks – `foo`.

In summary, our optimizer, which implements the rewrites described `foo`, improves the rendering time significantly for real-world websites.

5.4 RQ2 Does the optimizer pay for itself

While our optimizer does speed-up the rendering time large, high traffic websites, this rendering time is meaningless if the time taken to optimize is slow. In this section we aim to show that our optimizer pays for itself i.e. the time to optimize a skp is always far smaller than the time taken to render the optimized skp. To measure optimziation time, we replicated nanobench’s benchmarking scheme we described in `foo`.

We first show optimization times in seperation to everrything else. Figure 9 is a scatter plot of the time taken to optimize a benchmark vs the number of Skia commands in the benchmark. There arer two important facts to note from the figure. First, like we claim in Section 4, the time taken to optimize is linear wrt to the size of the Skia program. Firthermore, note the range of optimization times we report. All optimization times are within $25\ \mu\text{s}$. This is significantly smaller than the time taken to render. To visualize this, we draw the CDF from the previous section, except instead of calculating speedups as between baseline and optimzied render time, we now factor in the optimization time.

Figure 10 is visually indistnguishable from Figure 8. We originally wanted to superimpose both plots to show the tiny difference between the two plots, but in reality such a plot was not very helpful given how similiar the two plots are. To sumamrize, we show our optimizer is linear with respect to the number of commands in the Skia program, and that the time taken to optimization is far smaller than the time taken to render.

6 RELATED WORK

7 CONCLUSION

REFERENCES

- [1] [n.d.]. Introducing the B3 JIT Compiler. <https://github.com/WebKit/WebKit/blob/3b231beb6f11e77b502bd58e3b7bbe90c5ff07d1/Source/JavaScriptCore/b3/B3InsertionSet.h#L44>. Accessed: 2026-02-06.

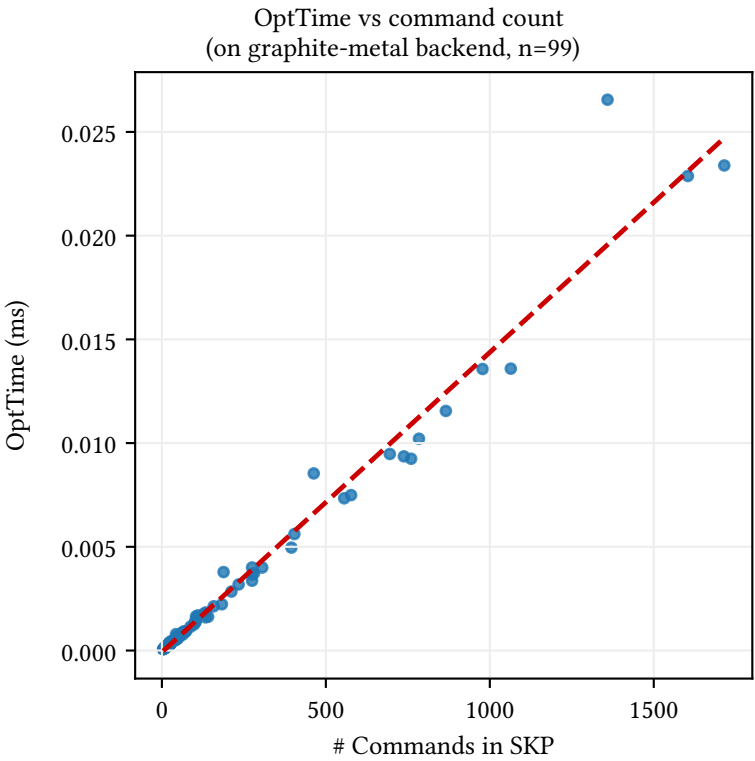


Fig. 9

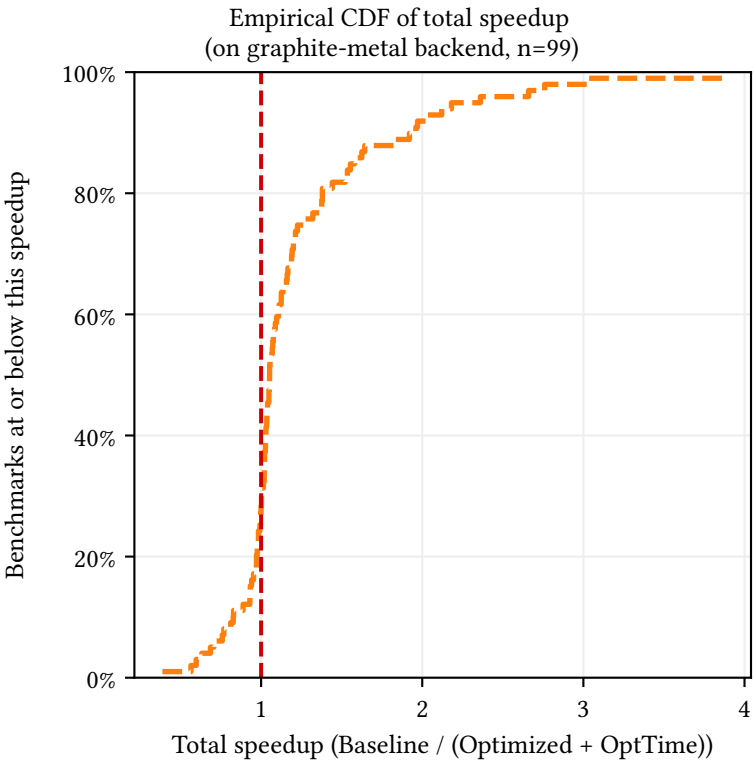


Fig. 10